Introduction to Combinational Circuits

5.1.1: Introduction to Combinational Circuits

A computer system is used to process discrete quantities of information. This information is represented using binary numbers and binary codes. A computer system is a very complex system that contains several logical circuits. Inputs applied and outputs generated are represented using variables. These variables take truth values which are governed by electric signals. In general, logical circuits fall into two categories:

1. Combinational circuits
2. Sequential circuits

Figure 1 shows the classification of logic circuits.

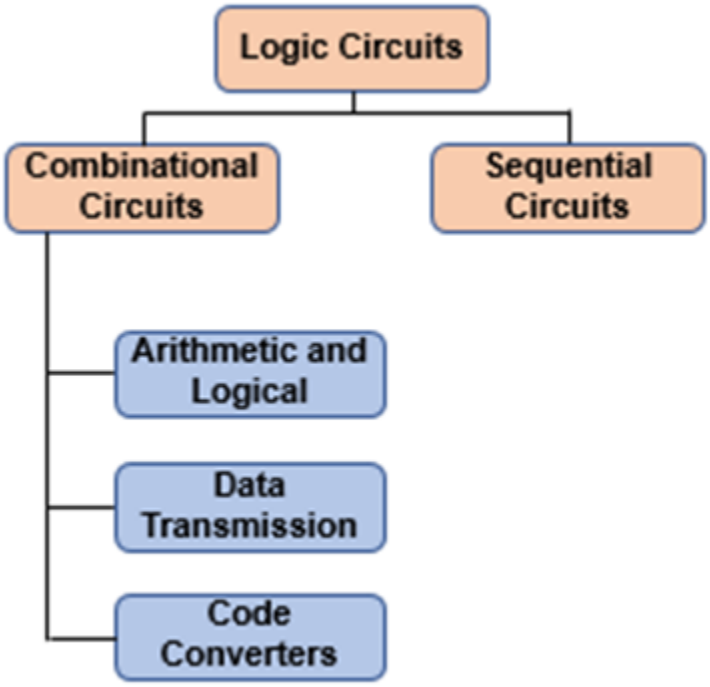


Figure 1: Classification of logic circuits.

A combinational circuit is one where the output at any time depends only on the present combination of inputs. It does not depend on the past state of the circuit. These kinds of circuits depend on the verbal statement of the problem. That is, the output variables are obtained from the given problem statement. For example, the problem statement for an adder circuit is to add two input bits and generate carry and sum as an output. From this problem statement, information regarding the number of inputs and the number of outputs is obtained, which can lead to a truth table that provides the minterms for simplifying and implementing the logic circuits. Combinational circuits are built using a network of logic gates. On the other hand, in a sequential circuit, the output depends not only on the present input but also on the past output state. Such circuits are said to have memory. Figures 2 and 3 show the combinational and sequential circuits.

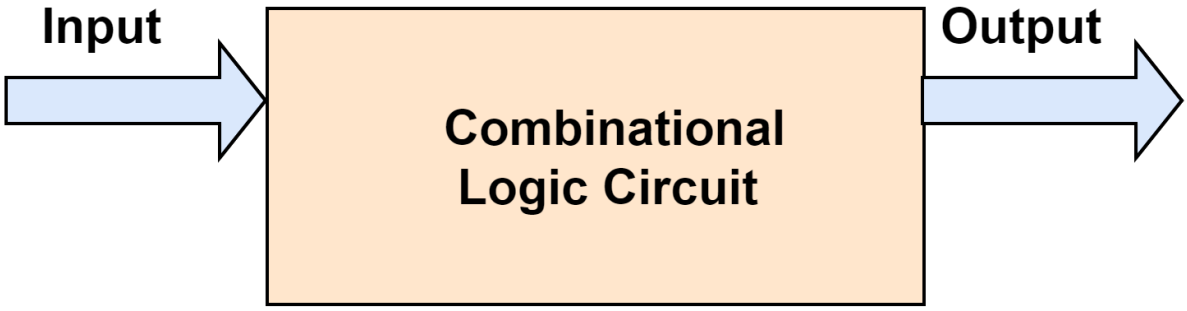


Figure 2: Combinational circuit

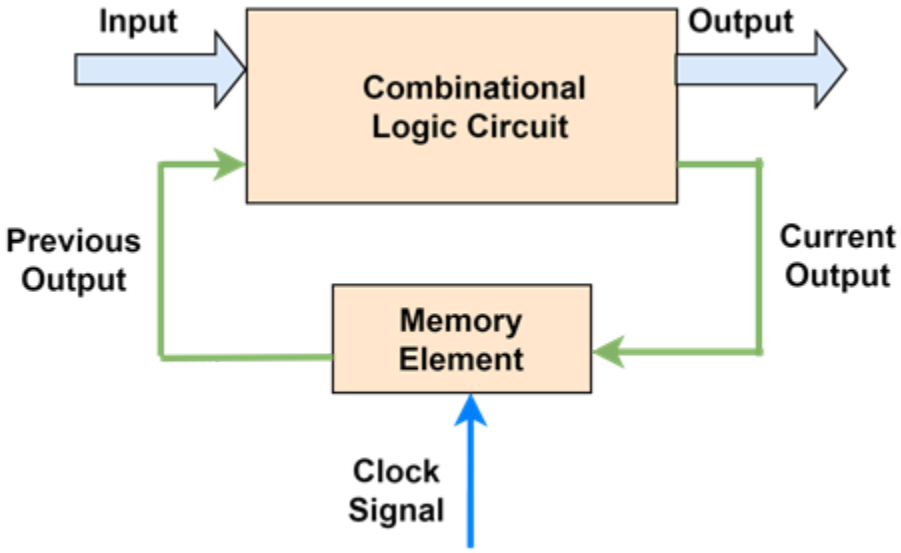


Figure 3: Sequential circuit.

Combinational circuits are categorized into three types, arithmetic and logical circuits, data transmission circuits, and code converters, based on the application domain where they are employed. Arithmetic and the logical unit is an integral part of a computer, which include an adder, subtractor, multiplier, sign extender, magnitude comparator, etc. Combinational circuits such as a multiplexer, demultiplexer, decoder, and encoder find their role in data transmission kind of applications such as communication systems, computer memory, etc. Combinational circuits also find their application in code converters such as binary, BCD, 7-segment, etc.

Combinational Circuits: Binary Adder

5.2.1: Half Adder

Adder is a very important logical circuit used in ALU and other parts of the computer. In this topic, we will see the design of a very basic type of adder known as a half adder. Adder is a digital circuit that performs the addition of binary numbers. The main application of adder is in ALU, that is, the Arithmetic and Logical Unit of a computer. It is used while computing an address of a memory location or table indices. It is also used in increment and decrement operations.

There are two types of adders:

1. Half adder
2. Full adder

Half adder is a combinational logic circuit for adding two single-bit numbers. It accepts two inputs and produces two outputs. The block diagram of the half adder is shown in Figure 1.

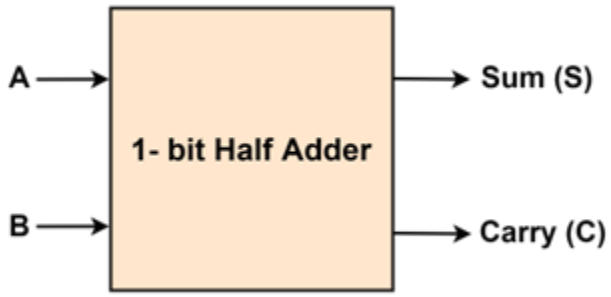


Figure 1: 1-bit half adder.

In general, the design of a combinational circuit involves four steps.

1. Identify the input and output variables.
2. Draw the truth table.
3. Determine the Boolean expression.
4. Draw the logic circuit.

The design procedure for half adder is as follows:

Step 1: Identify input and output variables.

There are two input variables, A and B. Both can take values 0 or 1. There are two outputs, Sum “S” and Carry “C”. Both these outputs produce 0 or 1 as output.

Step 2: Draw a truth table.

The truth table in Figure 2 has four columns: Two for inputs A and B and Two for outputs, Carry, and Sum.

Inputs		Outputs	
A	B	C(Carry)	S(Sum)
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Figure 2: Truth table of a half adder.

When A = 0, B = 0, Sum and Carry are equal to 0. When A = 0, B = 1, Sum is equal to 1, and Carry is equal to 0. When A = 1, B = 0, Sum is equal to 1, and Carry is equal to 0. When both A and B are equal to 1, Sum is equal to 0, and Carry is equal to 1.

Step 3: Obtain the expression for sum and carry outputs. The truth table shows that the carry output is high only when both inputs are high. For all other inputs, the carry is zero. So, this can be implemented using AND gate and the expression for carry

$C = AB$

As far as Sum output is concerned, when both the inputs are equal to zero, the sum is zero; otherwise, it is 1. So, the sum output has the behavior of the EXOR gate. Therefore, the expression for Sum

$S = A \oplus B$

Step 4: Draw a logic circuit. The logic circuit is implemented using two input EXOR gates and two input AND gates. The output of the EXOR gate represents the sum, and the output of the AND gate represents the carry shown in Figure 3.

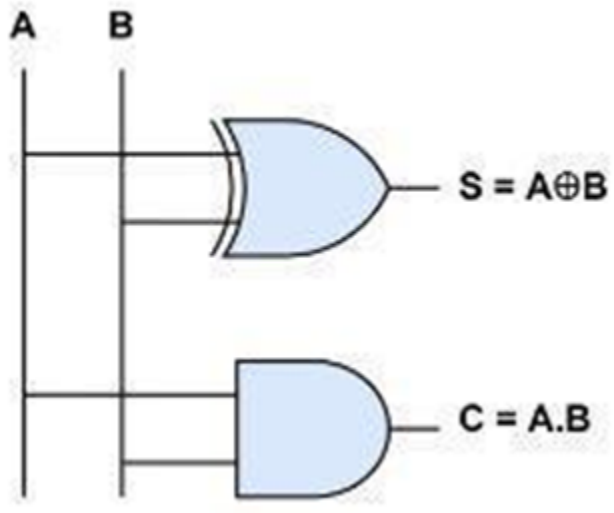


Figure 3: Logic circuit of a half adder.

Half adder has limited practical application because it is suitable only for single-bit addition. The main reason behind this is that it has only two inputs, and there is no provision to add a carry coming from the lower-order bits when multi-bit addition is performed.

5.2.2: Full Adder

To overcome the main drawback of a half adder, a full adder was developed. The full adder is capable of adding two binary digits plus a carry-in. A full adder is designed in four steps.

Step 1: Identify the input and output variables.

Figure 4 shows the block diagram of a full adder. 'A' and 'B' are the two input variables. These variables represent the two bits that are to be added. 'C_{in}' is the third input variable that represents the carry. This carry results from the previous addition at a lower significant position. The Sum S and Carry C_{out} are the output variables. All these input and output variables take truth values of 0 or 1.

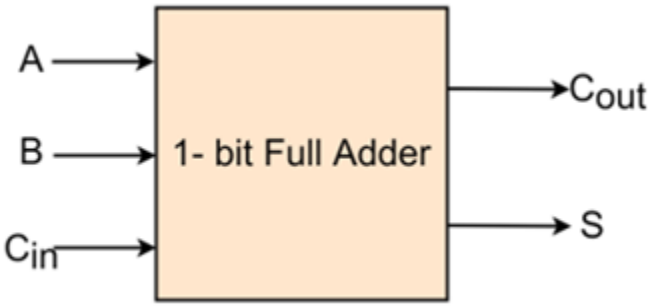


Figure 4: Block diagram of a full adder.

Step 2:Draw a truth table.

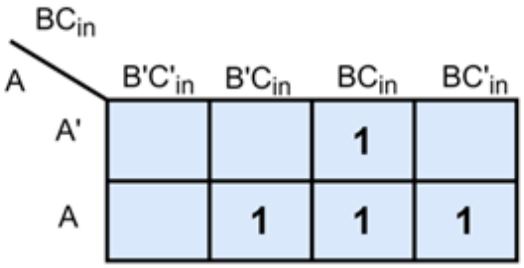
Three columns are for inputs A, B, and C_{in}, and two columns are for outputs C_{out} and S. Eight input combinations are possible for three inputs. When A = 0, B = 0 and C_{in} = 0, sum = 0. There is no carry generated. Hence, C_{out} is zero. When A = 0, B = 0 and C_{in} = 1, sum = 1, C_{out} is zero. When A = 0, B = 1 and C_{in} = 0, sum = 1, C_{out} is zero. When A = 0, B = 1, and C_{in} = 1, sum = 0, carry is generated, Hence, C_{out} is 1. Similarly, values for C_{out} and Sum for different values of A, B, and C_{in} are listed in Figure 5.

Inputs			Outputs	
A	B	C_{in}	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Figure 5: Truth table for a full adder.

Step 3: Determine the Boolean expression for C_{out} and S.

Since there are three inputs, three variables K-Map is used for simplification. For Cout, m3, m5, m6, and m7 are the min terms, and their position in the K-Map is as shown in Figure 6.



Min Term	Inputs			Outputs	
	A	B	C_{in}	C_{out}	S
m0	0	0	0	0	0
m1	0	0	1	0	1
m2	0	1	0	0	1
m3	0	1	1	1	0
m4	1	0	0	0	1
m5	1	0	1	1	0
m6	1	1	0	1	0
m7	1	1	1	1	1

Figure 6: K-Map for output Cout.

Simplification of K-Map results in,

Cout = ACin+ AB + BCin

The K-Map for sum output S is as shown in Figure 7. The final expression for sum, S is given by

S = AB'Cin' + A'B'Cin + ABCin + A'BC'in

= A

⊕

⊕ B

⊕

⊕Cin

Min Term	Inputs			Outputs	
	A	B	C_{in}	C_{out}	S
m0	0	0	0	0	0
m1	0	0	1	0	1
m2	0	1	0	0	1
m3	0	1	1	1	0
m4	1	0	0	0	1
m5	1	0	1	1	0
m6	1	1	0	1	0
m7	1	1	1	1	1

Figure 7: K-Map for sum output.
Step 4: Draw a logic diagram.
The implementation of the full adder is shown in Figure 8. To implement C_{out}, we need three AND gates and one OR gate; to implement S, we need four input XOR gate.

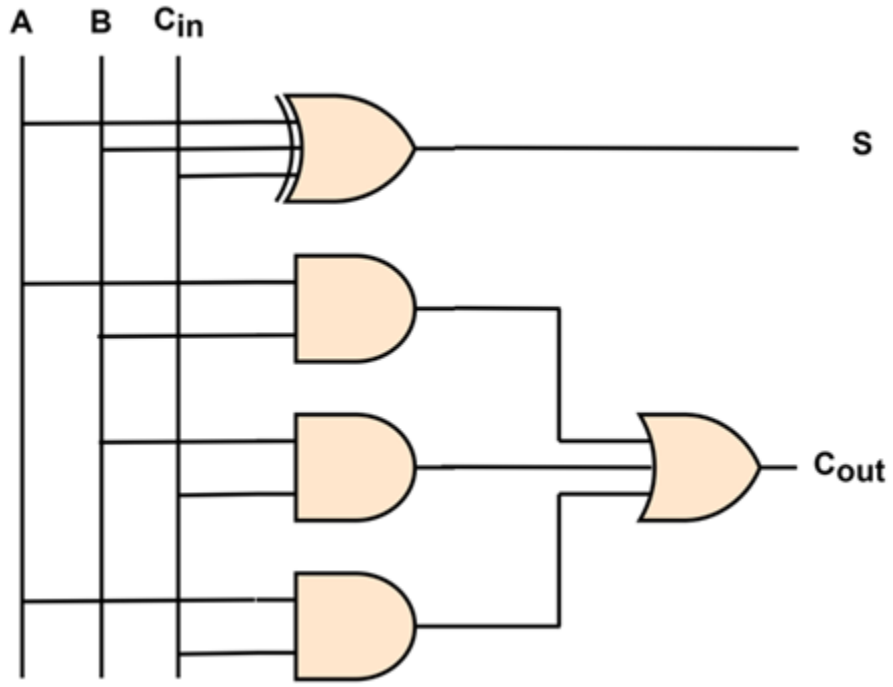


Figure 8: Logic circuit implementation of a full adder.

5.2.3: 4-bit Adder

Figure 9 shows the block diagram of a four-bit adder that adds two 4-bit numbers, A and B.

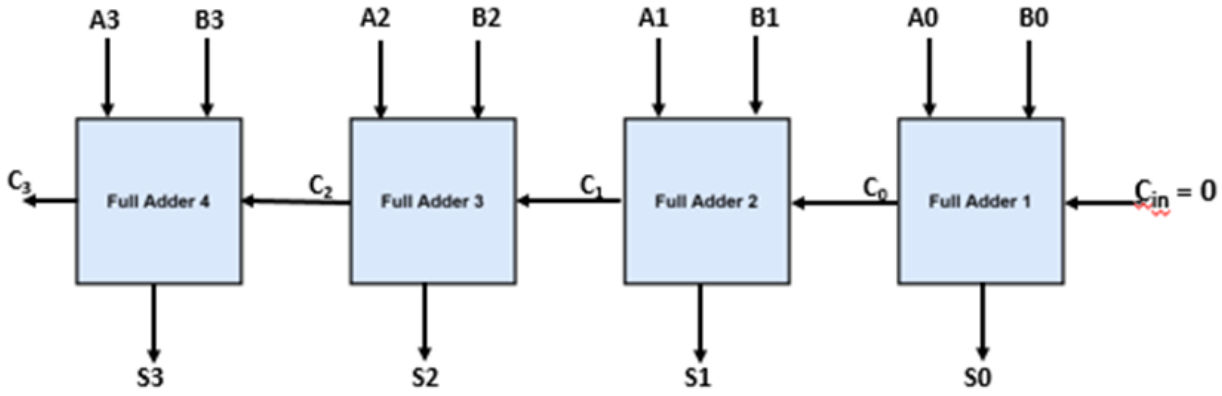


Figure 9: 4-bit adder.

Full adder has three inputs A, B, and C_{in} and two outputs S and C_{out}. A0 and B0 are LSBs, and A3 and B3 are MSBs. The addition starts from the Least Significant Bits, shown in Figure 10. In this position, C_{in} is set to 0. Every bit position, Sum and Carry is computed. When bits A0, B0, and C_{in} are added, the sum will be S0, and carry generated will be C0. This carry is fed to the next bit addition.

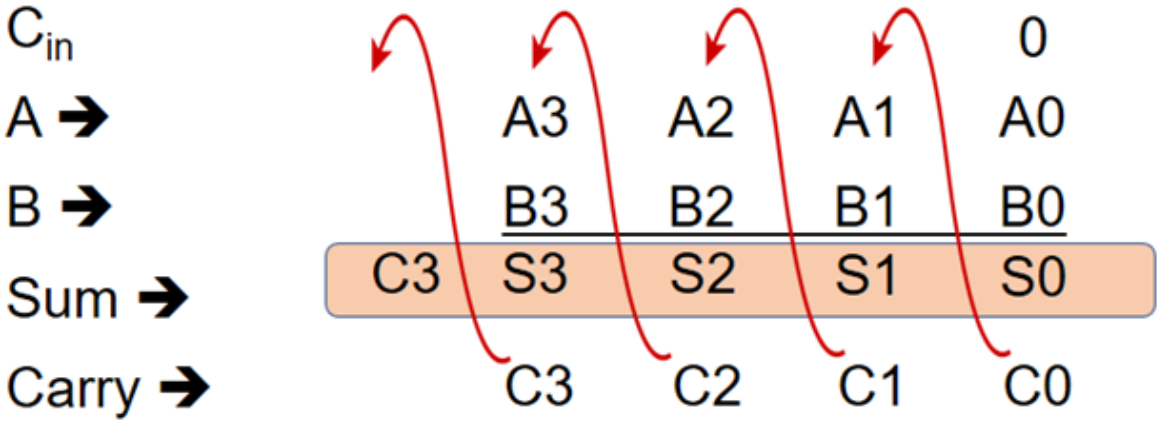


Figure 10: Process of 2, 4-bit binary addition using a full adder.
Adding A1, B1, and C0 produces a sum S1 and carry C1. This carry C1 is fed as the next C_{in}. Next, A2, B2, and C1 are added, which produces S2 and C2 as sum and carry, respectively. C2 is fed to the last bit position. Adding A3, B3, and C2 produces a sum S3 and Carry C3. Since this is the last addition, carry generated is added to the result. Hence, the final result will contain five bits C3, S3, S2, S1, and S0.
Since there are four bits in a number, four full adders are needed and connected in cascaded form. This type of adder is also known as a parallel adder since inputs are applied to all the full adders in parallel.
It is very easy to build 8-bit adder using a 4-bit adder. We just need to cascade them. C_{in} of the first 4-bit adder should be set at 0, C_{out} of the first 4-bit adder should be connected to C_{in} of the second 4-bit adder.

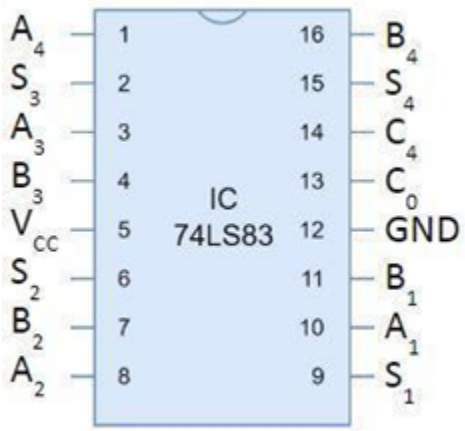


Figure 11: Pin diagram of IC 74LS83.
Figure 11 shows the commercially available 4-bit adder IC 74LS83. It is a 16-pin chip with 4-bit A input, 4-bit B input, and 4-bit Sum S output. C_0 and C_{out} represent carry-in and carry-out. Note that inputs are numbered as A_1, A_2, A_3, A_4 and B_1, B_2, B_3, B_4 where A_1, B_1 are LSBs and A_4, B_4 are is MSBs. Apart from these, there are two more pins, VCC and GND, which will be connected to 5V and OV, respectively.

Binary Subtractor and Comparator

5.3.1: Half Subtractor

Subtractor is a combinational circuit that is used to perform the subtraction of two bits. Similar to the adder, the main application of the Subtractor is in ALU. It is also used while accessing the table, computing memory addresses, etc. There are two types of Subtractors. Half Subtractor and Full Subtractor.

Figure 1 shows a block diagram of a half subtractor, a logic circuit used for subtracting one binary digit from another. For example, while performing X-Y, the subtractor accepts two inputs, X and Y, where X is minuend and Y is subtrahend. It produces two outputs, B and D, where B is a borrow and D is a difference. While designing the half subtractor, four steps are followed.

Step 1: Identify the input and output variables.

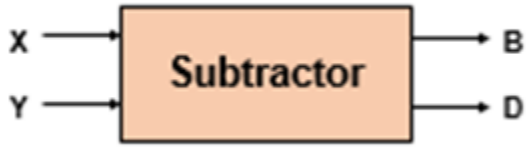


Figure 1: Block diagram of a half adder.

Two input variables, X and Y, can take truth values 0 or 1. Two output variables, Difference D and Borrow B. Both D and B can be 0 or 1.

Step 2: Draw the Truth Table.

Inputs		Outputs	
X	Y	B (Borrow)	D (Difference)
0	0	0	0
0	1	1	1
1	0	0	1
1	1	0	0

Figure 2: Truth table of a half subtractor.

Figure 2 shows the truth table of a half subtractor. When X = 0, Y = 0, the difference D is 0, and borrow B is 0. When X = 0, Y = 1, D is 1, with a B equals 1. When X = 1, Y = 0, D is 1 and B is 0. When X = 1, Y = 1, D is 0 and B is 0.

Step 3: Determine the Boolean expression.

B output is 1 only when X = 0 and Y = 1. This can be expressed as

B = X'Y.

Difference D output is at one for two input combinations. That is, when X = 0, Y= 1, and when X= 1, Y= 0. The behavior of the D output is the same as that of an EXOR gate. So, the Boolean expression for D output is

D =X

⊕

⊕Y.

Step 4: Draw the logic circuit.

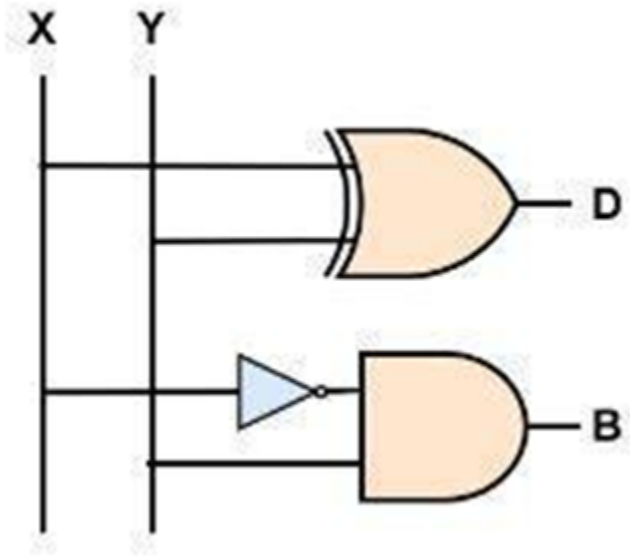


Figure 3: Logic circuit for half subtractor.

Figure 3 shows the logic circuit of a half subtractor. Implementation of a half subtractor requires three gates. The output of the exclusive OR gate is D output. The borrow part of the subtractor is implemented using NOT and AND gates.

Like a half adder, a half subtractor is also designed to work with two single bits. Hence, it is not suitable for implementing a multi-bit subtractor where we need to consider the borrow part as well.

5.3.2: Full Subtractor

A full subtractor is a combinational circuit designed to eliminate the limitation of a half subtractor. It has three inputs and two outputs. A full subtractor, shown in Figure 4, is designed in four steps.

Step 1: Identify the input and output variables.

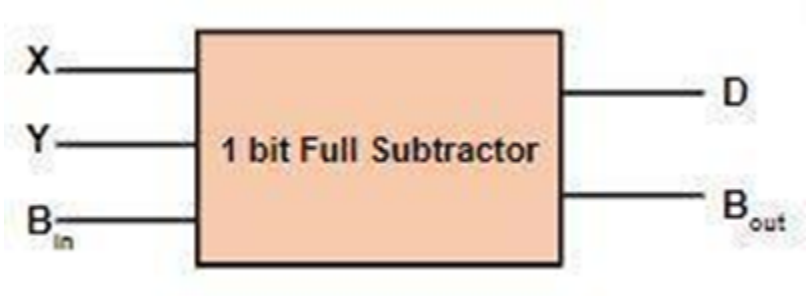


Figure 4: Block diagram of a full subtractor.

X and Y are the two input variables. X is minuend and Y is subtrahend. The third input is a borrow, B_{in}, which is generated during the subtraction operation on the previous least significant bits. D and B_{out} are two outputs generated by the subtractor. D is the difference output generated as a result of $X - Y - B_{in}$. B_{out} is the borrow output.

Step 2: Draw a truth table.

B_{out} and Sum for different values of X, Y, and B_{in} are shown in Figure 5.

Min Term	Inputs			Outputs	
	X	Y	B_{in}	B_{out}	D
m0	0	0	0	0	0
m1	0	0	1	1	1
m2	0	1	0	1	1
m3	0	1	1	1	0
m4	1	0	0	0	1
m5	1	0	1	0	0
m6	1	1	0	0	0
m7	1	1	1	1	1

Figure 5: Truth table for a full subtractor.

In the truth table, three columns are for inputs X, Y, and Bin, and two columns are for outputs D and Bout. When X = 0, Y = 0 and Bin = 0, D = 0. Since the borrow is not taken, Bout is zero. When X = 0, Y = 0, and Bin = 1, to compute D, we perform X-Y- Bin, that is, 0 – 0 – 1. This results in a difference D = 1 with the help of borrow. Hence, Bout is equal to 1. When X = 0, Y = 1 and Bin = 0, D = 1. In this case also a borrow is taken. Hence, Bout is 1. When X = 0, Y = 1 and Bin = 1, D = 0 and Bout =1. Bout and sum for other values of X, Y, and Bin are shown in Figure 19.

Step 3. Determine the Boolean expression for Bout and D.

Since there are three inputs, three variables K-Map is used for simplification. For Bout, m3, m5, m6, and m7 are the minterms and their position in the K-Map is as shown in Figure 6.

Figure 6: K-Map for Bout.

Boolean expression for Bout is given by,

Bout = X'Bin + X'Y + YBin

For D output, minterms are m1, m2, m4, and m7. The K-Map for D output is shown in Figure 7.

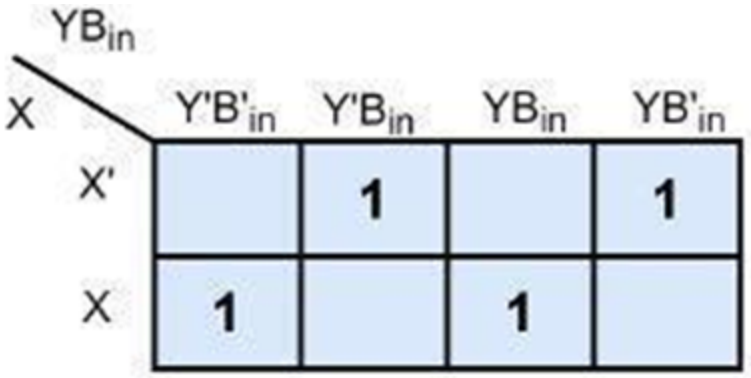


Figure 7: K-Map for D output.

There will be four terms and each term will have all three variables. The final expression for D output is,

D= XY'Bin' + X'Y'Bin + XYBin + X'YBin'

D = X

⊕

⊕ Y

⊕

⊕Bin

Step 4: Draw a logic diagram.

The logic circuit implementation of full adder is shown in Figure 8.

To implement Bout, 2 NOT gates, 3 AND gates, and one OR gate is needed, whereas to implement D, three input XOR gate is needed.

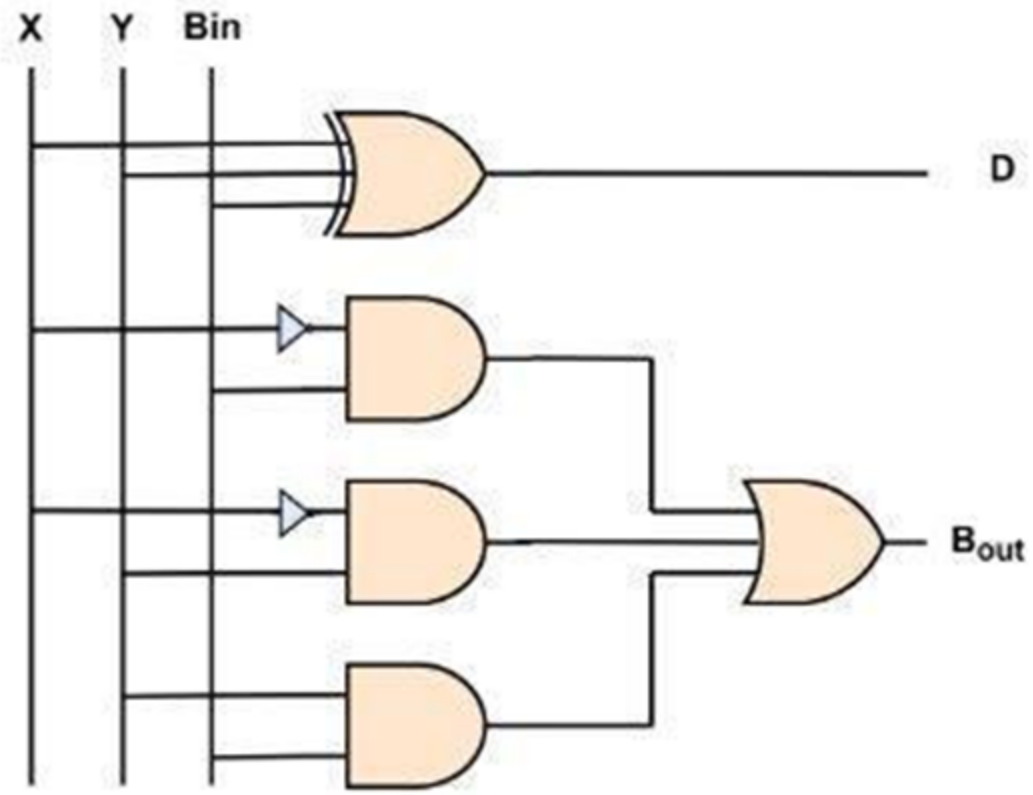


Figure 8: Logic circuit implementation of full subtractor.

3.3: Magnitude Comparator

A magnitude comparator is a combinational circuit that takes two numbers as input in binary form and determines whether one number is greater than, less than, or equal to another number. The device has two inputs and three outputs. Figure 9 shows a 1-bit magnitude comparator.

Figure 9: 1-bit magnitude comparator.

The design requires four steps to follow.

Step 1: Identify the input and output variables.

There are two input variables, X and Y, and three output variables, X>Y, X<Y, and X = Y.

Step 2: Draw the truth table.

Inputs		Outputs		
X	Y	X>Y	X = Y	X < Y
0	0	0	1	0
0	1	0	0	1
1	0	1	0	0
1	1	0	1	0

Figure 10: 1-bit comparator.

As shown in Figure 10, the truth table has five columns. Two for input variables and three for output variables. The output X>Y is 1 when X = 1 and Y = 0. For other combinations of X and Y, the output X>Y is 0. The output X=Y is 1 when X and Y are the same. The X<Y is 1, when X = 0 and Y = 1.

Step 3: Determine the Boolean expression.

The X>Y output is 1 for X = 1 and Y = 0. The Boolean expression is

X>Y □**XY'**.

The output is X = Y is 1 when X and Y are zero, and X and Y are 1. So, the Boolean expression is given

X=Y □ (**X'Y' + XY**).

The output X<Y is 1 when X = 0 and Y = 1. The Boolean expression for this condition is

X<Y □**X'Y**.

Step 4. Draw the logic diagram.

The logic circuit for the 1-bit magnitude comparator is shown in Figure 11.

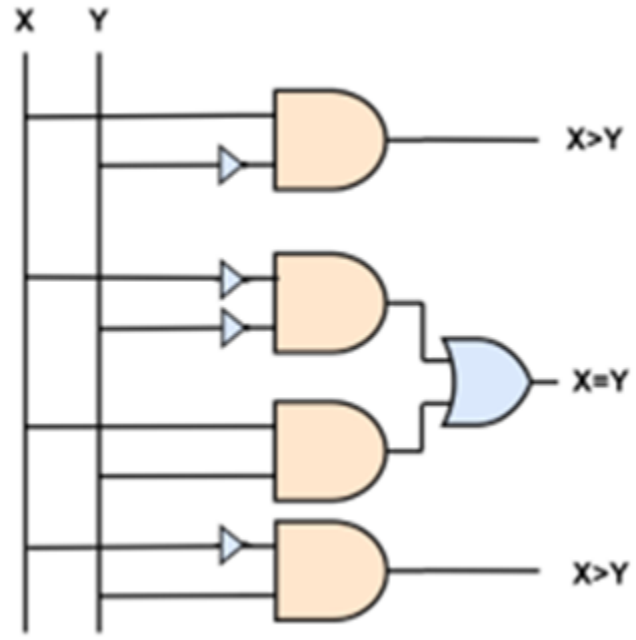


Figure 11: Logic circuit of 1-bit magnitude comparator.

The X > Y function is implemented using NOT and AND gates. NOT gate is used to complement Y input. The logic diagram for X=Y output uses two AND gates, two NOT gates, and one OR gate.

The output of the first AND gate is X'Y'. The output of the second AND gate is XY. Finally, the OR gate gives X'Y' + XY. The logic diagram is for X<Y output implements X'Y and requires NOT and AND gates.

Combinational Circuits: Data Transmission Circuits

5.4.1: Decoder

In a digital system, information is represented using binary codes. Most digital systems require the decoding of data. For example, digital display, digital to analog converters, memory addressing, etc. The decoder is a combinational circuit that converts an n-bit input code into 2^n outputs. Each output line will be activated for only one of the possible input combinations. Usually, decoders are designated as an n:m lines decoder, where n is the number of input lines and m is the number of output lines which is equal to 2^n.

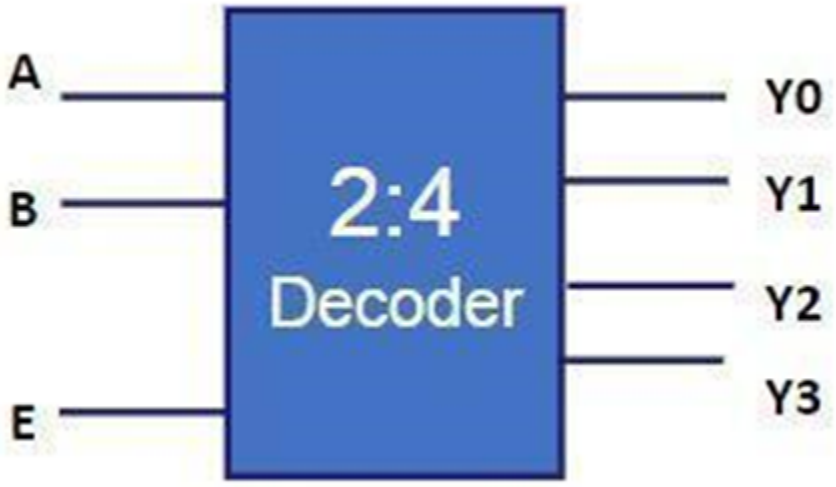


Figure 1: Block diagram of 2:4 decoder.

Figure 1 shows the 2:4 decoder with one enable signal E, two inputs A and B, and four outputs: Y3, Y2, Y1, Y0. The truth table for the same is shown in Figure 2.

Enable	Inputs		Outputs			
E	A	B	Y3	Y2	Y1	Y0
0	X	X	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

Figure 2: Truth table for 2:4 decoder.

When enable input E is low, the circuit is disabled. Irrespective of input status, all outputs will be at 0. The decoder circuit will be enabled when E input is set to 1. When A and B both are zero, output Y0 will be 1. All other outputs will be 0. When A = 0, B = 1, output Y1 will be 1. When A = 1, B = 0, output Y2 = 1. When A = 1, B =1, output Y3 = 1.
Boolean expressions for four outputs are as follows:
Y0 = E A' B'
Y1 = E A' B
y2 = E A B'
Y3 = E A B.
Logic circuit implementation is shown in Figure 3. The 2:4 decoder is implemented using two not gates and 4 AND gates.

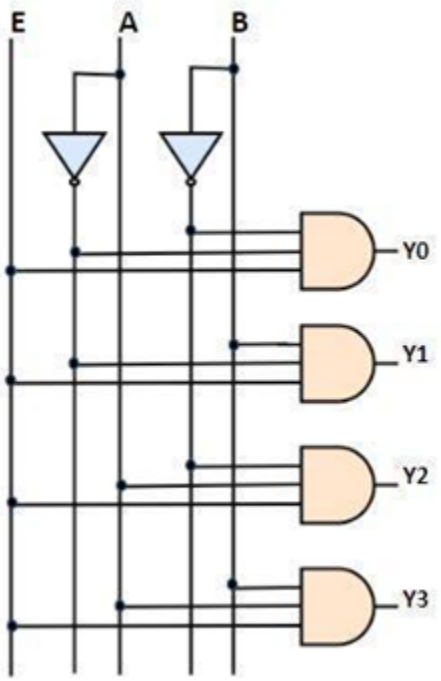


Figure 3: 2:4 decoder.
Another popular decoder is 3:8 decoder. This decoder consists of one enable input, three inputs, and 2_3, that is, eight outputs. Based on the input combination, one of the outputs will at logic 1. The logic circuit for the decoder is shown in Figure 4. It consists of three NOT gates and eight AND gates.

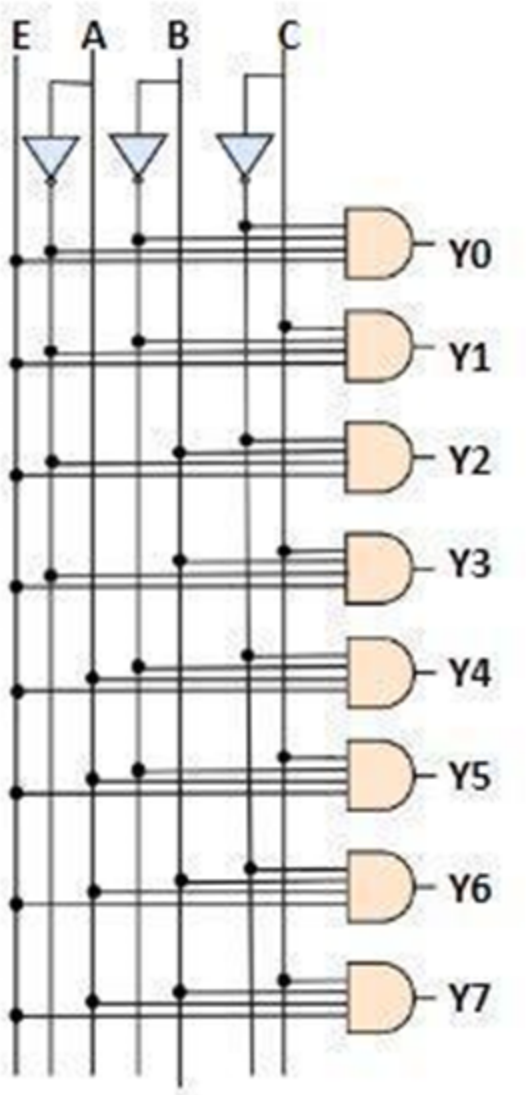


Figure 4: 3:8 decoder.

5.4.2: Encoder

An encoder is a combinational circuit with multiple inputs and outputs. Its working principle is exactly opposite to that of a decoder. The decoder has ‘n’ input lines and 2^n output lines, whereas the encoder has 2^n input lines and n output lines. This encoder encodes input information from 2^n inputs into an n-bit code. Figure 5 shows the block diagram of an encoder.

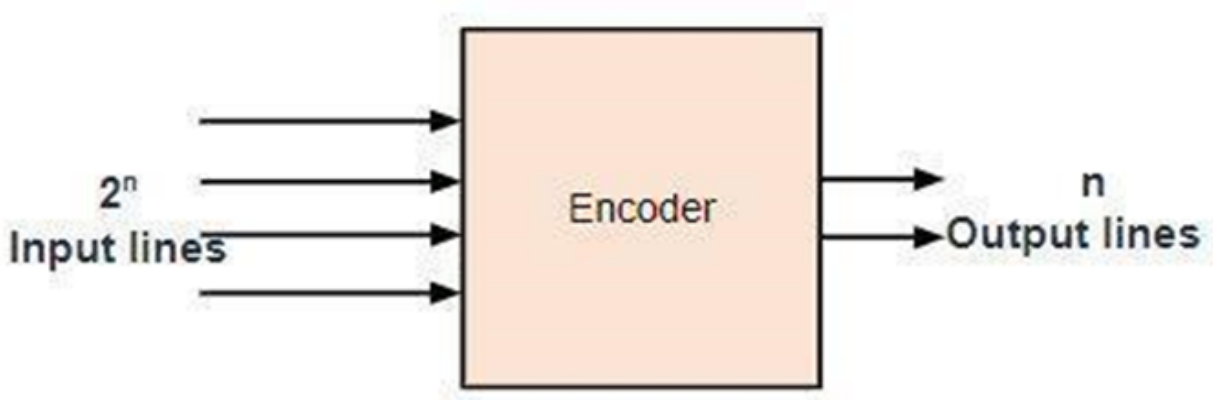


Figure 5: Block diagram of an encoder.

In 4:2 encoder, there are four inputs X_3, X_2, X_1, and X_0 and two outputs Y_1 and Y_0. At any time, only one of these four inputs can be '1' to get the respective binary code at the output. Figure 6 shows the truth table of the 4:2 encoder.

Input				Output	
X_3	X_2	X_1	X_0	Y_1	Y_0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

Figure 6: Truth table for a 4:2 encoder.

When X_0 is 1, the code generated will be $Y_0 = 0$, $Y_1 = 0$. When X_1 is 1, the code generated will be $Y_0 = 0$, $Y_1 = 1$. When X_2 is 1, the code generated will be $Y_0 = 1$, $Y_1 = 0$. When X_3 is 1, the code generated will be $Y_0 = 1$, $Y_1 = 1$. Boolean functions for each output are given by

$Y_0 = X_3 + X_1$

$Y_1 = X_3 + X_2$

It can be observed that X_0 has no effect on the output. Logic circuit implementation is shown in Figure 7. The implementation requires two OR gates. These two OR gates help in encoding four inputs with two-bit code.

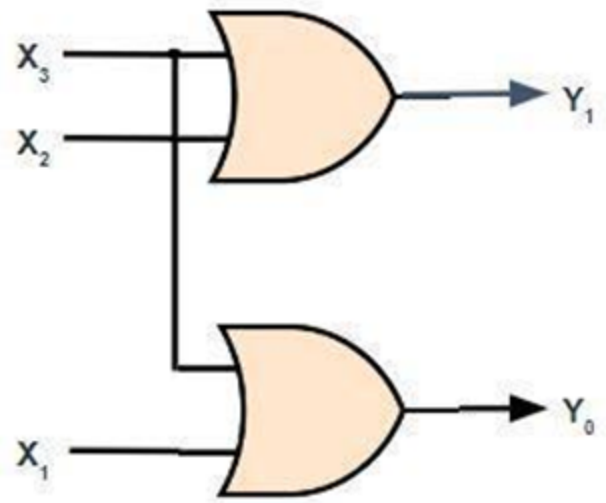


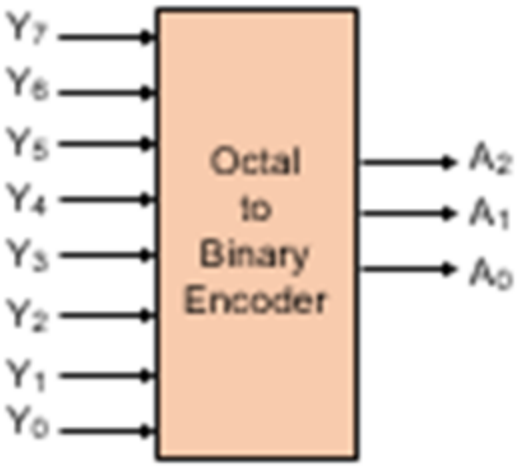
Figure 7: Logic implementation of 4:2 encoder.

5.4.3: Applications of Decoder and Encoder

A decoder circuit converts a binary code on the input to a single output representing the numeric value of the code. In contrast, an encoder circuit performs the reverse operation of the decoder. In practical scenarios such as data communication, we need to encode data at the transmitter, and this encoded data needs to be decoded later at the receiver end. In this reading, the application of the decoder and encoder is discussed. In the first application, an octal number is encoded into binary code using an encoder, whereas in the second application, binary code is decoded into octal using a decoder circuit.

Design an Octal to Binary Encoder:

The octal number system uses eight symbols that can be represented using a 3-bit binary code. Hence, an encoder that can take eight inputs representing eight symbols and produce 3-bit binary code as output is needed. The 8:3 encoder will have eight inputs Y_7 to Y_0, and three outputs A_2, A_1, and A_0. At any time, only one of these eight inputs can be ‘1’ to get the respective binary code. The octal to binary encoder and its truth table is shown in Figure 8.



Input								Output		
Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0	A_2	A_1	A_0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

Figure 8: Truth table for octal to binary encoder.

From the truth table, it is evident that the output A_2 becomes 1, if any of the digits Y_4, Y_5, Y_6, or Y_7 is one. Thus, the expression for A_2 can be written as

$A_2 = Y_4 + Y_5 + Y_6 + Y_7$

The output A_1 becomes 1 if any of the digits Y_2, Y_3, Y_6, or Y_7 is 1. Thus, the expression for A_1 is

$A_1 = Y_2 + Y_3 + Y_6 + Y_7$

Similarly, the output A_0 becomes 1 if any of the digits Y_1, Y_3, Y_5, or Y_7 is 1. The Boolean expression for A_0 is

$A_3 = Y_1 + Y_3 + Y_5 + Y_7$

The logic circuit for the octal to binary encoder is shown in Figure 9. The logic circuit is implemented using three OR gates.

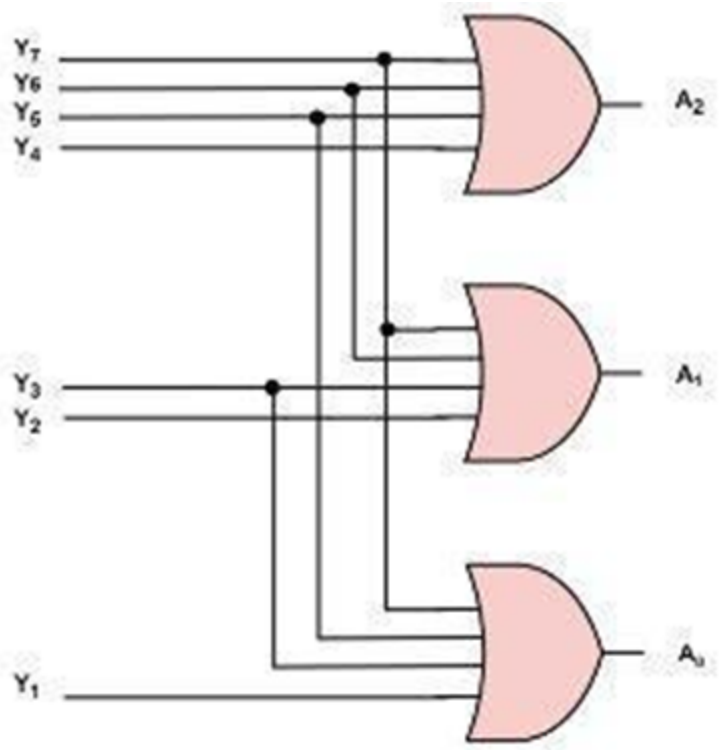
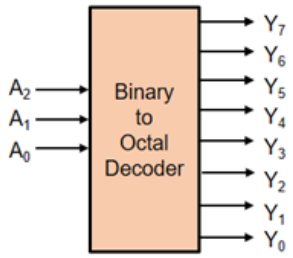


Figure 9: Octal to binary encoder logic circuit

A decoder circuit converts a binary code on the input to a single output representing the numeric value of the code. The binary-to-octal decoder converts three binary bits code into a 1-of-8 output. This decoder has three inputs, A2, A1, and A0, representing a binary code of three bits, and the eight outputs, Y7 to Y0, are the octal numbers 7 through 0. The block diagram and truth table for the binary to octal decoder are shown in Figure 10. For each input binary code, only one output line is at the logic 1 state. For example, when A2 A1 A0 is 0 0 0, Y0 will be at logic one state. A2' A1' A0 gives the Boolean expression for Y0'. When A2 A1 A0 is 0 0 1, Y1 will be at logic 1 state. The corresponding Boolean expression for Y1 is A2' A1' A0. The Boolean expression for other combinations of input binary code is shown in Figure 10.



Input			Output								
A_2	A_1	A_0	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0	
0	0	0	0	0	0	0	0	0	0	1	→ $A_2' A_1' A_0'$
0	0	1	0	0	0	0	0	0	1	0	→ $A_2' A_1' A_0$
0	1	0	0	0	0	0	0	1	0	0	→ $A_2' A_1 A_0'$
0	1	1	0	0	0	0	1	0	0	0	→ $A_2' A_1 A_0$
1	0	0	0	0	0	1	0	0	0	0	→ $A_2 A_1' A_0'$
1	0	1	0	0	1	0	0	0	0	0	→ $A_2 A_1' A_0$
1	1	0	0	1	0	0	0	0	0	0	→ $A_2 A_1 A_0'$
1	1	1	1	0	0	0	0	0	0	0	→ $A_2 A_1 A_0$

Figure 10: Block diagram and truth table for binary to octal decoder.

The binary-to-octal decoder shown in Figure 11 is implemented with three inverters and eight three-input AND gates.

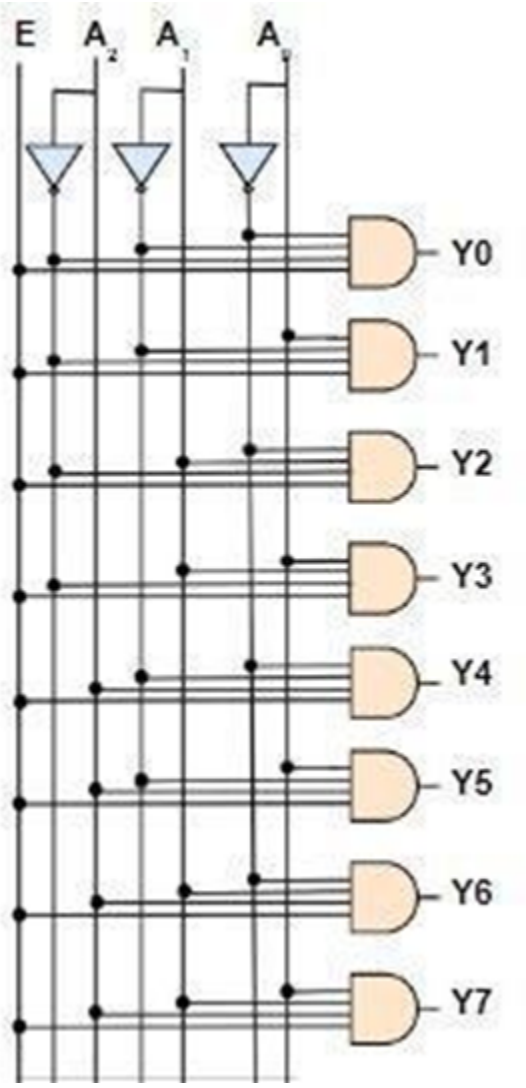


Figure 11: Logic circuit for binary to octal decoder.

5.4.4: Multiplexer

Multiplexer is a combinational circuit, in short, mux. Mux is analogous to a single pole-n-throw switch and follows many to one principle. It selects one of several inputs and then feeds it through to a single output. Figure 12 shows a multiplexer that consists of several input lines and one output line.

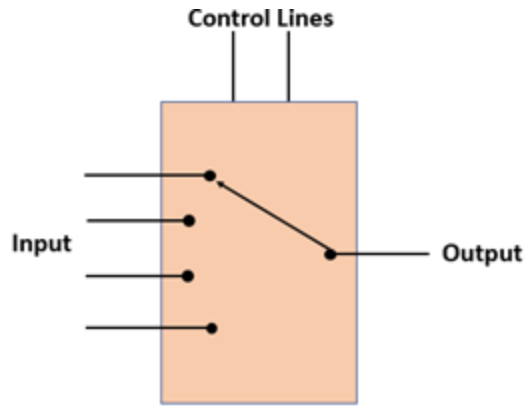


Figure 12: Block diagram of a multiplexer.

Mux is also known as a data selector, as the control lines determine which data input is selected. The control line is also known as the select line.

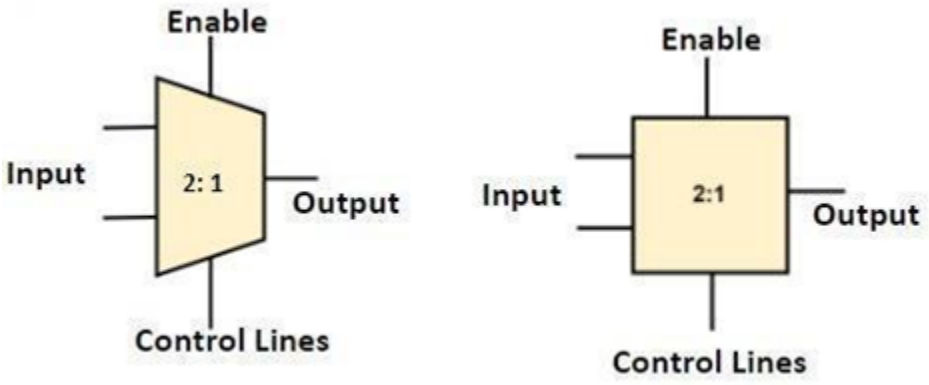


Figure 13: 2x1 multiplexer symbol.

Multiplexer symbol is shown in Figure 13. Designers use a square box or trapezoids to represent a multiplexer.

There is a relationship between control lines and input lines. Let M be the number of input lines and N be the number of control lines. Then,

$2^N = M$.

If a multiplexer has eight input lines, then it would require three control lines.

Multiplexors are very popular because of three main advantages.

1. The number of wires is reduced. For example, consider a data communication network. The signal from four different devices is combined using a multiplexer and sent on a single communication channel. Without a multiplexer, four channels or four long wires are needed to send these signals.
2. Reduced circuit complexity and cost.
3. It can be used to implement combinational circuits such as adder, comparator, etc. There is no need to use K-Map or Boolean simplification technique while implementing the circuit.

There are five popular multiplexers.

1. 2 to 1 multiplexers
2. 4 to 1 multiplexers
3. 8 to 1 multiplexers

4. 16 to1 multiplexers
5. 32 to 1 multiplexers

Figure 14 shows the block diagram, logic diagram, and truth table of a 2x1 multiplexer. In 2×1 multiplexer, there are only two inputs, i.e., A_0 and A_1, one selection line, S_0, and a single output, Y. Based on the value at select line S_0, one of the two inputs will be connected to the output. As you can see in the truth table, when S_0 is 0, A_0 is connected to the output, and when S_0 is 1, A_1 is connected to the output. Hence, the logic expression for output Y is given by

$Y = S_0' A_0 + S_0 A_1$

The logic circuit implementation requires two AND gates, one NOT gate, and one OR gate.

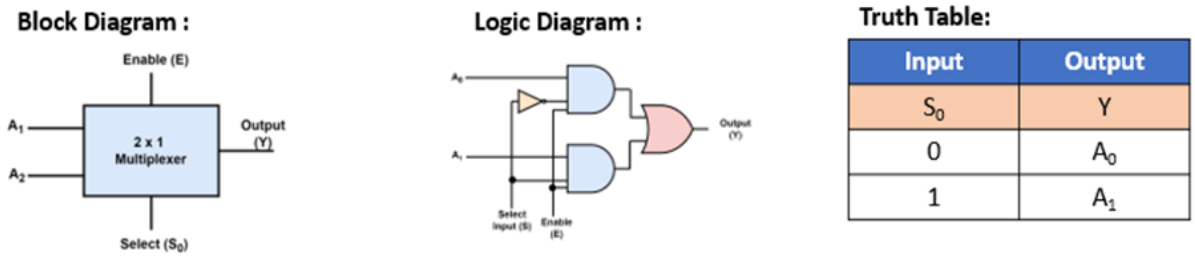


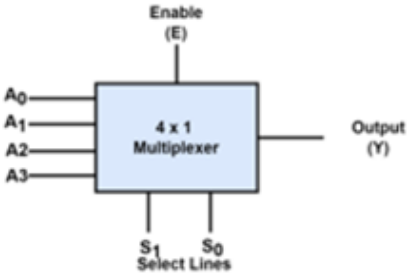
Figure 14: Block diagram, logic diagram, and truth table of 2x1 multiplexer.

Figure 15 show the block diagram, logic diagram, and truth table of 4x1 multiplexer. In 4x1multiplexer, there are four inputs, A_0, A_1, A_2, and A_3, two selection lines, S_0 and S_1, and a single output, Y. On the basis of the combination of inputs present at the selection lines S_0 and S_1, one of these four inputs is connected to the output. Boolean expression for 4x1 multiplexer is

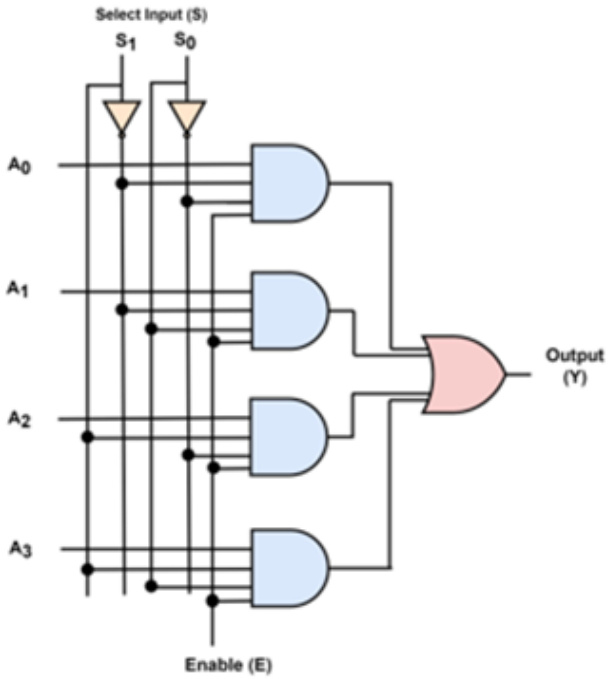
$Y = S_1' S_0' A_0 + S_1' S_0 A_1 + S_1 S_0' A_2 + S_1 S_0 A_3$

The logic implementation Boolean function requires four AND gates, two NOT gates, and one OR gate.

Block Diagram :



Logic Diagram :



Truth Table:

Inputs		Output
S_1	S_0	Y
0	0	A_0
0	1	A_1
1	0	A_2
1	1	A_3

Figure 15: Block diagram, logic diagram, and truth table of 4x1 multiplexer.

IC 74151 is a very popular high-speed 8x1 multiplexer. It provides one-of-eight data sources as a result of a unique three-bit binary code at the select inputs. It provides complementary and non-complementary outputs. IC 74153 has two 4x1 multiplexer.

5.4.5: Demultiplexer

Figure 16 shows a demultiplexer, in short, Demux, a combinational circuit with single-input and multiple-output lines. The information received from the single input lines is directed to one of the output lines based on the values set at the selection or control lines. The function of the demultiplexer is exactly opposite to the multiplexer. Multiplexers are called data selectors, whereas demultiplexers are data distributors because they transmit similar information obtained at the input to various outputs.

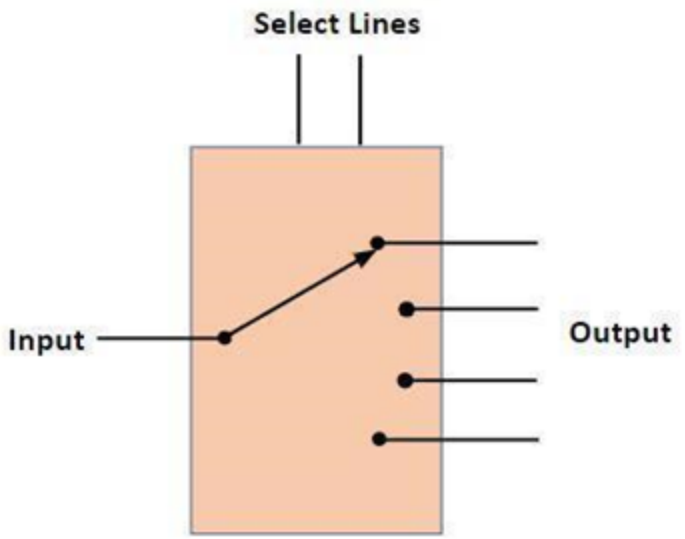


Figure 16: Demultiplexer.

Figure 17 shows the symbol of 1 to M demultiplexer. It has one input line and several output lines Some designers use a square box instead of trapezoid.

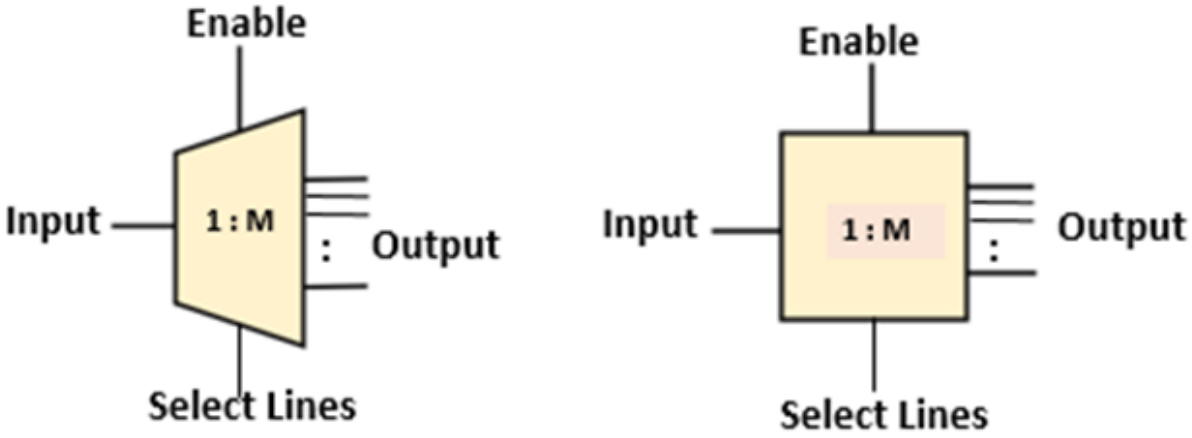


Figure 17: Symbol of demultiplexer.

There is a relationship between the number of select lines and output lines. Let M be the number of output lines and N be the number of select lines. Then,

$2^N = M$ or

$N = \log_2 M$

There are different types of demultiplexers available depending on the different output configurations like

- 1 to 2 Demultiplexer
- 1 to 4 Demultiplexer (IC 74139)
- 1 to 8 Demultiplexer (IC 74138)
- 1 to 16 Demultiplexer (IC 74154)

These demultiplexers are available in various IC packages. Popular ones are: IC 74139, a dual 1 to 4 Demux, IC 74138 is a 1 to 8 Demux, whereas IC 74154 is a 1 to 16 Demux.

Figure 18 shows the 1:2 demultiplexer, a basic demultiplexer with one input and two outputs. Usually, the input line is used to carry data. Hence, it is indicated as data line D. Since there are two output lines Y_0 and Y_1, we need one select signal S to control data flow from input to output. The truth table for this demultiplexer is as shown. When S equals zero, output Y_0 is selected, and data D is sent on this output line Y_0. Similarly, when S equals one, output Y_1 is selected and data D is sent on Y_1. The logic expression for

$Y_0 = S' D$

$Y_1 = S D$

Implementing these two logic expressions requires two AND gates and one NOT gate.

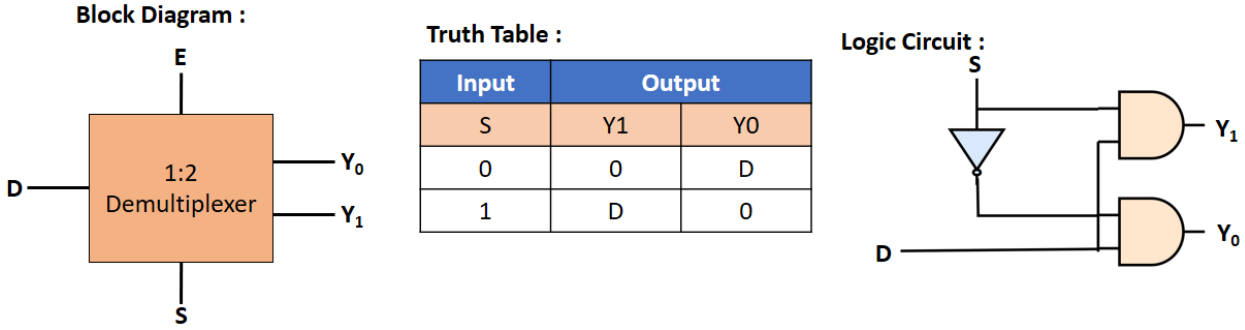


Figure 18: 1:2 Demultiplexer.

Figure 19 shows the block diagram of 1 to 4 demultiplexer. It has one data line D, four output lines, Y_0, Y_1, Y_2, and Y_3, and two select lines S_0 and S_1. The demultiplexer truth table is as shown. For each combination of select lines, one of the outputs will be connected to the input data line. Logic expression for four outputs and the corresponding logic expression is given by

$Y_0 = S_1' S_0' D$

$Y_1 = S_1' S_0 D$

$Y_2 = S_1 S_0' D$

$Y_3 = S_1 S_0 D$

1:4 demultiplexer can be implemented using three 1:2 demultiplexers. Data input is applied to Demux 0. Based on the value at S_1, either X_0 or X_1 will be selected. When S_1 is 0, X_0 will be selected, data D will be fed to Demux 1. When S_0 is 0, Y_0 is selected, and when S_0 is 1, Y_1 is selected. Similarly, when S_1 is 1, X_1 will be selected, and data D will be fed to Demux 2. When S_0 is 0, Y_2 is selected, and when S_0 is 1, Y_3 is selected.

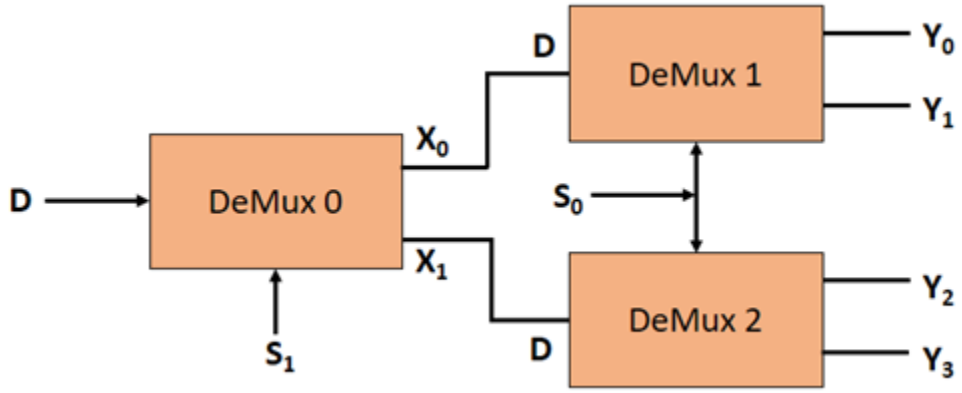


Figure 19: Implementation of 4:1 demultiplexer using 1:2 multiplexers.

5.4.6: Applications of Multiplexer and Demultiplexer

The multiplexer is one of the most widely used combinational circuits in digital design. It is a data selector. A multiplexer is many to one, which has many input lines but only one output. A demultiplexer is one-to-many, which has one input and many outputs.

One of the most common applications of a multiplexer is its use in implementing combinational logic Boolean functions. The technique is straightforward. To realize “n” variable Boolean function, we need two power n input multiplexers. The main advantage is a simplification of logic expression is not required. In most digital systems, data are processed parallel to achieve faster processing speeds. But when it comes to transmitting these data relatively large distances, this is done serially. The reason is that the parallel arrangement would require many lengthy transmission lines. Multiplexers can be used for parallel-to-serial conversion. Multiplexers are also used to fetch a bit of data from memory at a given memory address. Usually, the processor will have several multiplexers (MUX) controlling the data and address buses. Multiplexers are switches allowing the processor to select data from multiple data sources. Data communication between computers happens over a computer network. A multiplexer is used to combine and send the multiple data streams over a single medium. Similarly, telephone networks integrate multiple audio signals into a single transmission line with a multiplexer’s help. The multiplexer performs parallel to serial conversion at the transmitter side. At the receiving end, serial conversion is performed by the demultiplexer. Communication systems such as computer and telephone networks require at the receiving end demultiplexer circuit. Another important application of demultiplexer is in arithmetic and logical unit. In an Arithmetic logic unit (ALU), the output of the ALU can be stored in a storage unit, like multiple registers by using demultiplexer. In this process, the output of ALU is connected as an input to the demultiplexer, and the output of demultiplexer is connected to the registers to store the data.

The multiplexer can also be used as a logic element in the design of combinational circuits. The procedure is as follows:

- Identify the decimal number corresponding to minterms. The multiplexer input lines corresponding to these numbers are to be connected to logic 1 level
- All other input lines are to be connected to logic 0 level
- The inputs are to be applied to select lines.

Example: Implement the following Boolean expression using a multiplexer.

f (A,B,C) = m (0, 1, 3, 6, 7)

The given function is three variable functions and has five minterms 0, 1, 3, 6, and 7. So we need a multiplexer with three select lines. Use 8:1 multiplexer. 8:1multiplexer has eight input lines. Minterms 0, 1, 3, 6, and 7 correspond to inputs 0, 1, 3, 6, and 7 and connect them to Logic 1. Connect remaining input lines 2, 4, and 5 to logic 0. Function input variables A, B, and C should be connected to select lines. Figure 20 shows the implementation of Boolean expression.

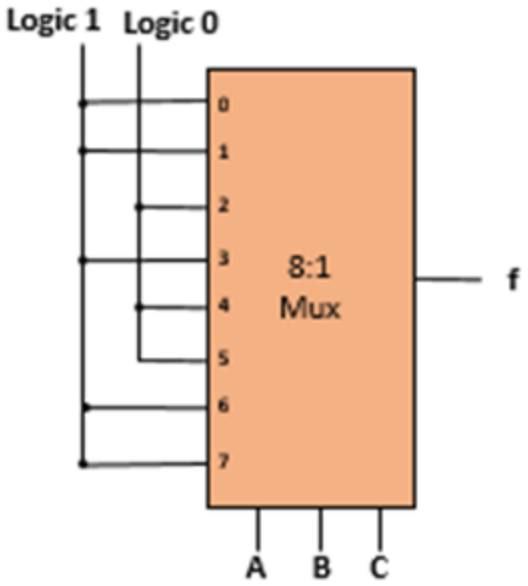


Figure 20: Logic circuit implementation of f (A,B,C) = m (0, 1, 3, 6, 7).